



Developer

AWS Developer Associate (DVA-C02)

Curso completo com 4 módulos

Desenvolvimento • Segurança • Implantação • Troubleshooting

CloudTrilhas — cloudtrilhas.com.br

Por Ricardo Simines Scopim

 **Developer — Domínio 01**

Desenvolvimento com Serviços AWS

Lambda, API Gateway, DynamoDB, S3, SQS, SNS, Step Functions e arquitetura event-driven. Este domínio representa 32% do exame.

AWS Lambda

Serviço de computação serverless que executa código em resposta a eventos sem provisionar ou gerenciar servidores.

Conceitos Fundamentais

Handler **ENTRY POINT**

Função que o Lambda chama para iniciar a execução. Recebe o evento e o contexto.

```
# Python
def handler(event, context):
    return {
        'statusCode': 200,
        'body': json.dumps('Hello')
    }

# Node.js
exports.handler = async (event, context) => {
    return { statusCode: 200, body: 'Hello' };
};
```

Configurações **LIMITES**

- Timeout: 1s a 15 min (padrão: 3s)
- Memória: 128 MB a 10.240 MB (64 MB increments)
- CPU: proporcional à memória alocada
- Disco (/tmp): 512 MB a 10.240 MB
- Pacote deploy: 50 MB (zip) / 250 MB (unzipped)
- Variáveis de ambiente: 4 KB total
- Payload síncrono: 6 MB
- Payload assíncrono: 256 KB
- Concurrent executions: 1000 (padrão por região)

Tipos de Invocação

Tipo	Comportamento	Retry	Exemplos
Síncrono	Espera resposta. Caller gerencia erros.	Nenhum (caller decide)	API Gateway, ALB, CloudFront, Cognito
Assíncrono	Retorna 202 imediatamente. Lambda gerencia retry.	2 retries (total 3 tentativas)	S3, SNS, EventBridge, CloudWatch Events
Event Source Mapping	Lambda faz polling da source. Processa em batches.	Retry até expirar ou sucesso	SQS, Kinesis, DynamoDB Streams, Kafka

Concorrência

Reserved Concurrency **LIMITE**

Reserva um número fixo de instâncias para a função. Garante capacidade mas também limita.

- Garante que a função sempre terá X instâncias disponíveis
- Limita a função a no máximo X execuções simultâneas
- Throttling (429) se exceder o limite
- Definir como 0 = desabilita a função

Provisioned Concurrency **WARM**

Mantém instâncias pré-inicializadas (warm). Elimina cold starts.

- Instâncias sempre prontas (sem cold start)
- Custo adicional (paga por instância provisionada)
- Pode usar Auto Scaling para ajustar
- Ideal para funções com latência crítica
- Funciona com aliases e versões publicadas

Layers, DLQ, Destinations, Versions e Aliases**Lambda Layers** **COMPARTILHAMENTO**

Pacotes de código/bibliotecas compartilhados entre funções.

- Até 5 layers por função
- Tamanho total (função + layers): 250 MB unzipped
- Montado em /opt
- Versionado independentemente
- Pode ser compartilhado entre contas

DLQ e Destinations **ERRO HANDLING**

Tratamento de falhas em invocações assíncronas.

DLQ (Dead Letter Queue):

- SQS ou SNS como destino de falhas
- Apenas para invocações assíncronas
- Envia após todos os retries falharem

Destinations (recomendado):

- Suporta sucesso E falha
- Destinos: SQS, SNS, Lambda, EventBridge
- Mais informações no payload que DLQ
- Pode encadear funções

Versions & Aliases DEPLOY

Controle de versão e roteamento de tráfego.

Versions:

- Snapshot imutável (código + config)
- \$LATEST = versão mutável (dev)
- Cada publish cria v1, v2, v3...

Aliases:

- Ponteiro nomeado para uma versão
- Ex: "prod" → v3, "dev" → \$LATEST
- Weighted alias: split traffic (canary)
 - prod: 90% → v3, 10% → v4
- Não pode referenciar outro alias



Para o exame: Destinations > DLQ (mais flexível). Provisioned Concurrency elimina cold starts. Event Source Mapping faz polling (SQS, Kinesis, DynamoDB Streams). Weighted aliases permitem canary deployments.

Amazon API Gateway

Serviço gerenciado para criar, publicar e gerenciar APIs em qualquer escala.

Tipos de API

Tipo	Protocolo	Características	Quando Usar
REST API	HTTP/HTTPS	Mais recursos (caching, API keys, usage plans, WAF, resource policies)	APIs complexas que precisam de todos os recursos
HTTP API	HTTP/HTTPS	Mais barato (~70% menos), menor latência, OIDC/OAuth2 nativo	APIs simples, proxies para Lambda/HTTP, custo menor
WebSocket API	WebSocket	Conexão bidirecional persistente, real-time	Chat, streaming, notificações em tempo real

Stages, Throttling e Caching

Stages AMBIENTES

Ambientes de deploy da API (dev, staging, prod).

- Cada stage = URL diferente
`https://api-id.execute-api.region.amazonaws.com/prod`
- Stage variables: variáveis por stage
 - Referenciar Lambda aliases: `${stageVariables.alias}`
- Canary deployments no stage
 - % do tráfego para nova versão
- Deploy history com rollback

Throttling RATE LIMIT

Proteção contra excesso de requisições.

Limites padrão (por região):

- 10.000 requests/segundo (steady-state)
- 5.000 requests burst

Níveis de throttling:

1. Account level (regional)
2. Stage level (usage plan)
3. Method level (per-method)

Resposta: HTTP 429 Too Many Requests

Authorizers

Tipo	Como Funciona	Caching	Quando Usar
Lambda Authorizer (Token)	Recebe token, retorna IAM policy	Sim (até 1h)	Tokens customizados, OAuth, SAML, lógica complexa
Lambda Authorizer (Request)	Recebe headers, query strings, context	Sim	Múltiplos parâmetros de autenticação
Cognito User Pool	Valida JWT token do Cognito	Não (validação local)	Autenticação com Cognito, sem lógica custom
IAM Authorization	SigV4 signing	N/A	Chamadas entre serviços AWS, cross-account

Integration Types e CORS

Integration Types

BACKEND

- **LAMBDA_PROXY** (mais comum):
 - Request inteiro passado ao Lambda
 - Lambda retorna statusCode + headers + body
 - Sem mapping templates
- **LAMBDA** (custom):
 - Usa mapping templates (VTL)
 - Transforma request/response
 - Mais controle, mais complexo
- **HTTP_PROXY**:
 - Passa request direto ao HTTP endpoint
 - Sem transformação
- **HTTP** (custom):
 - Com mapping templates
- **AWS_PROXY**:
 - Integração direta com serviços AWS
- **MOCK**:
 - Retorna resposta sem backend

CORS **CROSS-ORIGIN**

Cross-Origin Resource Sharing para chamadas de browsers.

Headers necessários:

- Access-Control-Allow-Origin
- Access-Control-Allow-Methods
- Access-Control-Allow-Headers

REST API:

- Habilitar CORS no console
- OPTIONS method (preflight)
- Configurar nos Integration Response

HTTP API:

- Configuração simplificada
- Suporte nativo a CORS

⚠ LAMBDA_PROXY:

- Lambda DEVE retornar os headers CORS
- API Gateway não adiciona automaticamente

API Gateway Caching

Cache **PERFORMANCE**

- Disponível apenas em REST API
- Cache por stage (0.5 GB a 237 GB)
- TTL: 0 a 3600 segundos (padrão: 300s)
- Pode invalidar com header:
Cache-Control: max-age=0
- Custo adicional por hora
- Pode ser por método específico
- Encrypt cache data: opcional

 **Para o exame:** HTTP API é mais barato e rápido que REST API, mas REST API tem mais recursos (caching, API keys, WAF). LAMBDA_PROXY é o tipo de integração mais usado. Com LAMBDA_PROXY, o Lambda deve retornar headers CORS manualmente.

Amazon DynamoDB

Banco de dados NoSQL serverless, totalmente gerenciado, com latência de milissegundos em qualquer escala.

Modelo de Dados

Chaves

PRIMARY KEY

Partition Key (PK):

- Obrigatória
- Determina a partição física
- Deve ter alta cardinalidade
- Hash function distribui dados

Sort Key (SK) – opcional:

- Permite múltiplos itens por PK
- Ordenação dentro da partição
- Permite range queries (begins_with, between)

Composite Key = PK + SK

- Combinação deve ser única
- Ex: PK=UserId, SK=OrderDate

Índices Secundários GSI / LSI

GSI (Global Secondary Index):

- PK e SK diferentes da tabela base
- Pode ser criado a qualquer momento
- Tem própria capacidade (RCU/WCU)
- Eventually consistent apenas
- Até 20 GSIs por tabela
- Projeção: ALL, KEYS_ONLY, INCLUDE

LSI (Local Secondary Index):

- Mesma PK, SK diferente
- Deve ser criado na criação da tabela
- Compartilha capacidade da tabela
- Suporta strongly consistent reads
- Até 5 LSIs por tabela
- Limite: 10 GB por partition key

Capacity Modes

Modo	Cobrança	Quando Usar	Características
Provisioned	RCU/WCU definidos	Tráfego previsível	Auto Scaling disponível, Reserved Capacity para desconto
On-Demand	Pay-per-request	Tráfego imprevisível, novo app	Sem planejamento de capacidade, 2.5x mais caro por request

RCU / WCU **CÁLCULO**

RCU (Read Capacity Unit):

- 1 RCU = 1 strongly consistent read/s (até 4 KB)
- 1 RCU = 2 eventually consistent reads/s (até 4 KB)
- Transactional: 2 RCUs por read (4 KB)

WCU (Write Capacity Unit):

- 1 WCU = 1 write/s (até 1 KB)
- Transactional: 2 WCUs por write (1 KB)

Exemplo: Item de 6 KB

- Strong read: $\text{ceil}(6/4) = 2$ RCUs
- Eventually: $\text{ceil}(6/4) / 2 = 1$ RCU
- Write: $\text{ceil}(6/1) = 6$ WCUs

DynamoDB Streams, TTL, DAX e Transactions

DynamoDB Streams **CDC**

Captura mudanças na tabela em tempo real (Change Data Capture).

Stream Views:

- KEYS_ONLY: apenas as chaves
- NEW_IMAGE: item após a mudança
- OLD_IMAGE: item antes da mudança
- NEW_AND_OLD_IMAGES: ambos
- Retém dados por 24 horas
- Processado por Lambda ou Kinesis
- Garante ordem por partition key
- Ideal para: replicação, triggers, analytics

TTL **EXPIRAÇÃO**

Remove itens automaticamente após timestamp expirar.

- Atributo com epoch timestamp (seconds)
- Sem custo adicional (não consome WCU)
- Deletado em até 48h após expiração
- Item expirado aparece em queries até deletar
- Pode filtrar com FilterExpression
- Deletions aparecem no DynamoDB Streams
- Ideal para: sessões, logs temporários, cache

DAX **CACHE**

DynamoDB Accelerator — cache in-memory para DynamoDB.

- Latência de microssegundos (vs ms)
- Compatível com API do DynamoDB
- Write-through cache
- Item cache: GetItem, BatchGetItem
- Query cache: Query, Scan results
- TTL padrão: 5 minutos
- Multi-AZ (mínimo 3 nodes recomendado)
- Não suporta: strongly consistent reads
- Ideal para: read-heavy, hot keys

Transactions ACID

Operações ACID em múltiplos itens/tabelas.

- TransactWriteItems: até 100 itens
- TransactGetItems: até 100 itens
- All-or-nothing (atomicidade)
- Consome 2x capacidade (RCU/WCU)
- Idempotent com ClientRequestToken
- Casos: transferências, pedidos, inventário

PartiQL:

- SQL-compatible query language
- SELECT, INSERT, UPDATE, DELETE
- Funciona com console, CLI e SDK
- Suporta transactions



Para o exame: GSI pode ser criado depois, LSI apenas na criação. DAX = cache de leitura (microsegundos). Streams retém 24h. TTL não consome WCU. Transactions consomem 2x capacidade. On-Demand é mais caro mas sem planejamento.

Amazon S3

Armazenamento de objetos com durabilidade de 99.999999999% (11 noves) e escalabilidade ilimitada.

Storage Classes

Classe	Disponibilidade	Caso de Uso	Retrieval
S3 Standard	99.99%	Dados acessados frequentemente	Imediato
S3 Intelligent-Tiering	99.9%	Padrão de acesso desconhecido	Imediato
S3 Standard-IA	99.9%	Acesso infrequente, acesso rápido	Imediato (custo por retrieval)
S3 One Zone-IA	99.5%	Dados recriáveis, 1 AZ	Imediato (custo por retrieval)
S3 Glacier Instant	99.9%	Arquivo com acesso imediato	Milissegundos
S3 Glacier Flexible	99.99%	Arquivo, acesso em minutos/horas	1-5 min (Expedited) a 3-5h (Bulk)
S3 Glacier Deep Archive	99.99%	Arquivo de longo prazo	12h (Standard) a 48h (Bulk)

Lifecycle, Versioning, Presigned URLs

Lifecycle Rules

AUTOMAÇÃO

Transition Actions:

- Mover entre storage classes
- Ex: Standard → IA após 30 dias
- Ex: IA → Glacier após 90 dias

Expiration Actions:

- Deletar objetos após X dias
- Deletar versões antigas
- Deletar incomplete multipart uploads

Filtros:

- Por prefixo (pasta)
- Por tags
- Por tamanho do objeto

Versioning HISTÓRICO

- Habilitado no nível do bucket
- Cada versão tem ID único
- Delete = adiciona "delete marker"
- Recuperar: deletar o delete marker
- Não pode ser desabilitado (apenas suspenso)
- MFA Delete: proteção extra
- Integra com Lifecycle (versões antigas)
- Custo: armazena todas as versões

Presigned URLs ACESSO TEMPORÁRIO

- URL com credenciais temporárias
- Upload ou download
- Expira após tempo definido
 - Console: até 12h
 - CLI: até 7 dias (168h)
- Herda permissões de quem gerou
- Ideal para: conteúdo privado, uploads diretos

Gerar presigned URL (CLI)

```
aws s3 presign s3://bucket/key \  
--expires-in 3600
```

Event Notifications TRIGGERS

Eventos:

- s3:ObjectCreated:*
- s3:ObjectRemoved:*
- s3:ObjectRestore:*

Destinos:

- SNS Topic
- SQS Queue
- Lambda Function
- EventBridge (todos os eventos)
- Filtro por prefixo e sufixo
- EventBridge: mais flexível, mais destinos
- Pode ter múltiplas notificações por bucket

Transfer Acceleration

S3 Transfer Acceleration PERFORMANCE

- Usa CloudFront edge locations para acelerar uploads
- Endpoint: `bucketname.s3-accelerate.amazonaws.com`
- Custo adicional por GB transferido
- Ideal para: uploads de longa distância, grandes arquivos
- Multipart upload: recomendado para arquivos > 100 MB (obrigatório > 5 GB)

 **Para o exame:** Presigned URLs herdam permissões do criador. Event Notifications → SNS, SQS, Lambda ou EventBridge. Versioning não pode ser desabilitado após habilitado. Multipart upload obrigatório acima de 5 GB.

Amazon SQS

Serviço de filas de mensagens totalmente gerenciado para desacoplamento de componentes.

Standard vs FIFO

Característica	Standard	FIFO
Throughput	Ilimitado	300 msg/s (3000 com batching)
Ordenação	Best-effort (pode desordenar)	Garantida (FIFO)
Duplicatas	At-least-once (pode duplicar)	Exactly-once (deduplication)
Nome da fila	Qualquer nome	Deve terminar com .fifo
Message Group ID	N/A	Obrigatório (ordena dentro do grupo)
Deduplication ID	N/A	Obrigatório (ou content-based dedup)

Configurações Importantes

Visibility Timeout lock

- Tempo que mensagem fica "invisível" após ser lida
- Padrão: 30 segundos
- Mínimo: 0s | Máximo: 12 horas
- Se consumer não deletar a tempo → mensagem reaparece
- `ChangeMessageVisibility`: estender timeout
- Deve ser > tempo de processamento

⚠ Se mensagens são processadas 2x:
→ Aumentar visibility timeout

DLQ DEAD LETTER QUEUE

- Fila para mensagens que falharam N vezes
- `maxReceiveCount`: tentativas antes do DLQ
- `Redrive policy`: configuração do DLQ
- Mesmo tipo (Standard DLQ para Standard, FIFO para FIFO)
- `Retention`: até 14 dias
- `Redrive to source`: reprocessar mensagens do DLQ

Boas práticas:

- Alarme no DLQ (CloudWatch)
- `Retention` do DLQ > `retention` da fila original

Long Polling EFICIÊNCIA

Short Polling (padrão):

- Retorna imediatamente (mesmo vazio)
- Mais requests = mais custo

Long Polling (recomendado):

- Espera até ter mensagem (ou timeout)
- `WaitTimeSeconds`: 1-20 segundos
- Reduz requests vazios
- Reduz custo
- Configurar na fila ou no `ReceiveMessage`

⚠ Sempre usar Long Polling em produção

Batch Operations**PERFORMANCE**

- `SendMessageBatch`: até 10 mensagens
- `ReceiveMessage`: até 10 mensagens (`MaxNumberOfMessages`)
- `DeleteMessageBatch`: até 10 mensagens
- Tamanho máximo por mensagem: 256 KB
- Batch total: 256 KB
- Reduz custos (cobrado por request)
- Extended Client Library: mensagens > 256 KB (usa S3)

Delay Queue:

- `DelaySeconds`: 0-900s (15 min)
- Mensagem invisível por X segundos após envio
- Diferente de `visibility timeout`

 **Para o exame:** FIFO garante ordem e exactly-once. Long Polling reduz custo. Visibility Timeout deve ser maior que o tempo de processamento. DLQ deve ser do mesmo tipo da fila. Extended Client Library para mensagens > 256 KB.

Amazon SNS

Serviço de mensagens pub/sub para comunicação entre serviços e notificações.

Topics, Subscriptions e Fan-Out

Topics & Subscriptions PUB/SUB

Tipos de Topic:

- Standard: alta throughput, best-effort ordering
- FIFO: ordenação garantida, deduplication

Protocolos de Subscription:

- HTTP/HTTPS (webhooks)
- Email / Email-JSON
- SQS (fan-out pattern)
- Lambda
- SMS
- Kinesis Data Firehose
- Platform application (push mobile)
- Até 12.5M subscriptions por topic
- Até 100.000 topics por conta

Fan-Out Pattern ARQUITETURA

SNS + SQS para processar a mesma mensagem de formas diferentes.

SNS Topic

```

├─ SQS Queue A → Service A (email)
├─ SQS Queue B → Service B (analytics)
└─ SQS Queue C → Service C (archive)
  
```

Benefícios:

- Desacoplamento total
- Cada consumer processa independentemente
- Retry independente por fila
- Adicionar consumers sem alterar publisher

SNS FIFO + SQS FIFO:

- Ordenação end-to-end garantida
- Fan-out com ordem

Message Filtering FILTRO

Filtrar mensagens por subscription usando filter policies.

Filter Policy (JSON):

```
{
  "eventType": ["order_placed"],
  "store": ["store-a", "store-b"],
  "price": [{"numeric": [">=", 100]}]
}
```

- Aplicado por subscription
- Sem filtro = recebe tudo
- Reduz processamento desnecessário
- Filtro por atributos da mensagem
- Suporta: exact match, prefix, numeric, exists, anything-but, IP address

 **Para o exame:** SNS = push (entrega imediata). SQS = pull (consumer busca). Fan-out = SNS → múltiplas SQS. Message Filtering evita processamento desnecessário nos subscribers.

AWS Step Functions

Orquestração visual de workflows serverless usando state machines.

Standard vs Express

Característica	Standard	Express
Duração máxima	1 ano	5 minutos
Execução	Exactly-once	At-least-once (async) / At-most-once (sync)
Taxa de execução	2.000/s	100.000/s
Preço	Por transição de estado	Por execução + duração + memória
Histórico	90 dias no console	CloudWatch Logs
Caso de uso	Workflows longos, orquestração	Alto volume, IoT, streaming, ETL rápido

Tipos de Estado

States

TIPOS

- Task: executa trabalho (Lambda, ECS, etc.)
- Choice: decisão condicional (if/else)
- Parallel: execução paralela de branches
- Map: iteração sobre array (loop)
- Wait: pausa por tempo definido
- Pass: passa input para output (debug/transform)
- Succeed: finaliza com sucesso
- Fail: finaliza com erro

Error Handling RESILIÊNCIA

Retry:

- `ErrorEquals`: tipos de erro
- `IntervalSeconds`: tempo entre retries
- `MaxAttempts`: máximo de tentativas
- `BackoffRate`: multiplicador exponencial

Catch:

- `ErrorEquals`: tipos de erro para capturar
- `Next`: estado de fallback
- `ResultPath`: onde colocar o erro

Erros predefinidos:

- `States.ALL`: qualquer erro
- `States.Timeout`: timeout
- `States.TaskFailed`: falha na task
- `States.Permissions`: sem permissão

Service Integrations INTEGRAÇÕES

Padrões de integração:

- `Request Response`: chama e espera resposta
- `Run a Job (.sync)`: espera job completar
 - ECS, Glue, Batch, CodeBuild
- `Wait for Callback (.waitForTaskToken)`:
 - Pausa até receber token de callback
 - Ideal para aprovações humanas

Serviços integrados:

- Lambda, ECS, Fargate, Batch
- DynamoDB, SQS, SNS
- Glue, SageMaker, EMR
- API Gateway, EventBridge

 **Para o exame:** Standard = workflows longos (até 1 ano). Express = alto volume e curta duração (até 5 min). Retry + Catch para error handling. `.waitForTaskToken` para aprovações

humanas. Map state para processar arrays.

 Developer — Domínio 02

Segurança

IAM, Cognito, KMS, Secrets Manager, SSM Parameter Store, criptografia, autenticação e autorização. Este domínio representa 26% do exame.

AWS IAM (Identity and Access Management)

Controle de acesso granular para recursos AWS. Fundamento de segurança para todo o ecossistema.

Tipos de Policy

Tipo	Onde é Anexada	Gerenciamento	Caso de Uso
AWS Managed	Users, Groups, Roles	AWS mantém	Permissões comuns (ReadOnlyAccess, AdminAccess)
Customer Managed	Users, Groups, Roles	Você mantém	Permissões customizadas da organização
Inline	Diretamente no principal	Você mantém	Relação 1:1 estrita (não reutilizável)
Resource-based	No recurso (S3, SQS, Lambda)	Você mantém	Cross-account access, acesso direto ao recurso
Permission Boundary	Users, Roles	Você mantém	Limitar permissões máximas (delegação segura)
SCP (Service Control Policy)	AWS Organizations (OU/Account)	Org admin	Guardrails para toda a organização

Policy Evaluation Logic

Evaluation Order DECISÃO

1. Explicit Deny → DENY (sempre vence)
2. SCP → se não permite, DENY
3. Resource-based policy → se permite, ALLOW
4. Permission Boundary → se não permite, DENY
5. Session Policy → se não permite, DENY
6. Identity-based policy → se permite, ALLOW
7. Implicit Deny (padrão)

Regra de ouro:

- Deny SEMPRE vence Allow
- Sem permissão explícita = Deny implícito
- Resource-based pode conceder cross-account

Policy Structure JSON

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "AllowS3Read",
    "Effect": "Allow",
    "Action": [
      "s3:GetObject",
      "s3:ListBucket"
    ],
    "Resource": [
      "arn:aws:s3:::my-bucket",
      "arn:aws:s3:::my-bucket/*"
    ],
    "Condition": {
      "StringEquals": {
        "aws:RequestedRegion": "us-east-1"
      }
    }
  }]
}
```

IAM Roles, STS e Cross-Account

IAM Roles ASSUME ROLE

- Credenciais temporárias (não permanentes)
- Trust Policy: quem pode assumir a role
- Permission Policy: o que a role pode fazer
- Duração: 1h a 12h (padrão: 1h)

Casos de uso:

- EC2 Instance Profile
- Lambda Execution Role
- Cross-account access
- Federation (SAML, OIDC)
- Service-linked roles

AWS STS TOKENS

Security Token Service – credenciais temporárias:

- AssumeRole: assume role na mesma ou outra conta
- AssumeRoleWithSAML: federation com SAML
- AssumeRoleWithWebIdentity: federation com OIDC (Cognito Identity Pool é preferido)
- GetSessionToken: MFA para usuário IAM
- GetCallerIdentity: "quem sou eu?"

Retorna:

- AccessKeyId
- SecretAccessKey
- SessionToken (obrigatório nas chamadas)
- Expiration

Cross-Account Access MULTI-CONTA

Opção 1: Resource-based Policy

- Adicionar conta B na policy do recurso
- Não precisa assumir role
- Principal mantém suas permissões originais

Opção 2: AssumeRole (recomendado)

1. Conta A cria role com trust policy para conta B
2. Conta B assume a role via STS
3. Recebe credenciais temporárias da conta A

⚠ Resource-based: não perde permissões originais

⚠ AssumeRole: assume APENAS as permissões da role

Permission Boundaries LIMITE

- Define o MÁXIMO de permissões que um user/role pode ter
- Effective permissions = Identity policy $\cap$ Boundary
- Não concede permissões por si só
- Ideal para delegação segura:
 - Admin cria boundary
 - Dev cria roles dentro do boundary
 - Dev não pode escalar privilégios

Exemplo:

- Boundary permite: s3:*, dynamodb:*
- Identity policy permite: s3:*, ec2:*
- Effective: apenas s3:* (interseção)

🎯 **Para o exame:** Explicit Deny sempre vence. Permission Boundary = interseção com identity policy. Cross-account com resource-based policy mantém permissões originais. STS retorna credenciais temporárias.

Amazon Cognito

Serviço de identidade para aplicações web e mobile — autenticação, autorização e gerenciamento de usuários.

User Pools vs Identity Pools

Característica	User Pools	Identity Pools (Federated Identities)
Função	Autenticação (quem é você?)	Autorização (o que você pode fazer na AWS?)
Retorna	JWT Tokens (ID, Access, Refresh)	Credenciais AWS temporárias (STS)
Caso de uso	Sign-up, sign-in, MFA, social login	Acesso direto a serviços AWS (S3, DynamoDB)
Providers	Próprio, Google, Facebook, SAML, OIDC	Cognito User Pool, Google, Facebook, SAML, OIDC
Integração	API Gateway (Cognito Authorizer), ALB	IAM Roles para acesso a recursos AWS

Authentication Flows e Tokens

Auth Flows FLUXOS

USER_SRP_AUTH:

- Secure Remote Password (recomendado)
- Senha nunca trafega pela rede

USER_PASSWORD_AUTH:

- Envia username + password diretamente
- Menos seguro, mais simples

CUSTOM_AUTH:

- Lambda triggers para lógica customizada
- Desafios personalizados (CAPTCHA, OTP)

ADMIN_USER_PASSWORD_AUTH:

- Server-side (backend)
- Requer credenciais admin

Refresh Token:

- Obtém novos ID/Access tokens
- Sem re-autenticação do usuário
- Validade: 1 dia a 3650 dias

Tokens JWT

ID Token:

- Identidade do usuário
- Claims: email, name, phone, custom attributes
- Usado para personalização no frontend
- Validade: 5 min a 1 dia

Access Token:

- Autorização para APIs
- Claims: scopes, groups
- Usado no header Authorization: Bearer
- Validade: 5 min a 1 dia

Refresh Token:

- Obtém novos tokens sem re-login
- Validade: 1 dia a 10 anos
- Pode ser revogado

⚠ ID Token = quem é | Access Token = o que pode

MFA & Security PROTEÇÃO

MFA:

- TOTP (Google Authenticator)
- SMS (menos seguro)
- Pode ser obrigatório ou opcional

Advanced Security:

- Adaptive authentication
- Compromised credentials detection
- Account takeover protection
- Risk-based MFA

Lambda Triggers:

- Pre sign-up: validação customizada
- Post confirmation: welcome email
- Pre token generation: custom claims
- Custom message: personalizar emails
- Migrate user: migração de legacy

 **Para o exame:** User Pool = autenticação (JWT). Identity Pool = autorização (credenciais AWS). Fluxo comum: User Pool autentica → Identity Pool fornece credenciais AWS. API Gateway usa Cognito Authorizer com User Pool tokens.

AWS KMS (Key Management Service)

Gerenciamento centralizado de chaves de criptografia para proteger dados em repouso e em trânsito.

Tipos de CMK (Customer Master Key)

Tipo	Gerenciamento	Rotação	Caso de Uso
AWS Owned	AWS (invisível)	Varia	Serviços AWS internamente (SSE-S3)
AWS Managed	AWS (visível, aws/service)	Automática (1 ano)	Serviços AWS com sua chave (aws/s3, aws/ebs)
Customer Managed	Você gerencia	Configurável (opcional)	Controle total, cross-account, auditoria

Envelope Encryption e Key Policies

Envelope Encryption PADRÃO

Padrão usado para criptografar dados maiores que 4 KB.

Processo:

1. GenerateDataKey → retorna:
 - Plaintext data key (para criptografar)
 - Encrypted data key (para armazenar)
2. Criptografa dados com plaintext data key
3. Descarta plaintext data key da memória
4. Armazena encrypted data key junto aos dados

Decrypt:

1. Envia encrypted data key ao KMS
2. KMS retorna plaintext data key
3. Usa plaintext data key para decriptar dados

- ⚠ KMS só criptografa até 4 KB diretamente
- ⚠ Para dados maiores: SEMPRE envelope encryption

Key Policies & Grants

[ACESSO](#)

Key Policy (obrigatória):

- Toda CMK tem uma key policy
- Define quem pode usar/gerenciar a chave
- Sem key policy = ninguém acessa (nem root)
- Default: permite root account gerenciar

Grants:

- Permissões temporárias e granulares
- Programáticas (via API)
- Não precisam alterar key policy
- Ideal para: acesso temporário, delegação
- Podem ser revogados (RevokeGrant)

IAM Policy + Key Policy:

- Ambos devem permitir para funcionar
- Key policy deve permitir IAM policies
(statement com "Enable IAM User Permissions")

Data Key Caching PERFORMANCE

AWS Encryption SDK:

- Cache de data keys para reutilização
- Reduz chamadas ao KMS (custo + latência)
- LocalCryptoMaterialsCache

Configurações:

- Max age: tempo máximo no cache
- Max bytes encrypted: limite de dados
- Max messages encrypted: limite de mensagens

Quando usar:

- Alto volume de encrypt/decrypt
- Latência é crítica
- Custo de KMS API calls é relevante

⚠ Trade-off: segurança vs performance

 **Para o exame:** Dados > 4 KB = Envelope Encryption (GenerateDataKey). Key Policy é obrigatória. Customer Managed Key para controle total e cross-account. AWS Encryption SDK para data key caching.

Secrets Manager vs SSM Parameter Store

Dois serviços para armazenar configurações e segredos, com características diferentes.

Característica	Secrets Manager	SSM Parameter Store
Propósito	Segredos (senhas, API keys, tokens)	Configurações e segredos
Rotação automática	Sim (Lambda built-in para RDS, Redshift, DocumentDB)	Não nativo (precisa Lambda + EventBridge)
Criptografia	Sempre criptografado (KMS)	Opcional (SecureString com KMS)
Versionamento	Sim (automático)	Sim (labels: latest, v1, v2)
Cross-account	Sim (resource policy)	Não nativo (precisa assumir role)
Custo	\$0.40/segredo/mês + \$0.05/10K API calls	Standard: gratuito Advanced: \$0.05/parâmetro/mês
Tamanho máximo	64 KB	Standard: 4 KB Advanced: 8 KB
Hierarquia	Não	Sim (/app/prod/db/password)
Integração CloudFormation	Dynamic reference	Dynamic reference + resolve

Secrets Manager ROTAÇÃO

Rotação automática:

- Lambda function rotaciona o segredo
- Built-in para: RDS, Redshift, DocumentDB
- Custom Lambda para outros serviços
- Schedule: dias ou cron expression

Uso no código:

```
import boto3

client = boto3.client('secretsmanager')

response = client.get_secret_value(
    SecretId='prod/myapp/database'
)

secret = json.loads(response['SecretString'])
```

⚠ Cache o segredo no código (não buscar a cada request)

Parameter Store HIERARQUIA

Tipos de parâmetro:

- String: texto simples
- StringList: lista separada por vírgula
- SecureString: criptografado com KMS

Hierarquia:

```
/myapp/  
  /prod/  
    /db/host  
    /db/password (SecureString)  
    /api/key (SecureString)  
  /dev/  
    /db/host  
    /db/password
```

GetParametersByPath:

- Busca todos os parâmetros de um path
- Recursive: busca subpaths
- WithDecryption: decripta SecureStrings

Tiers:

- Standard: 4 KB, 10K params, gratuito
- Advanced: 8 KB, 100K params, policies

 **Para o exame:** Precisa de rotação automática → Secrets Manager. Configurações simples e gratuitas → Parameter Store. Hierarquia de parâmetros → Parameter Store. Segredos de banco com rotação → Secrets Manager.

Criptografia em Repouso e em Trânsito

Padrões de criptografia para proteger dados em diferentes estados.

Encryption at Rest **ARMAZENAMENTO**

S3:

- SSE-S3: chave gerenciada pela AWS (padrão)
- SSE-KMS: chave KMS (auditoria, controle)
- SSE-C: chave fornecida pelo cliente
- Client-side: criptografa antes de enviar

DynamoDB:

- AWS owned key (padrão, gratuito)
- AWS managed key (aws/dynamodb)
- Customer managed key (CMK)

EBS:

- AES-256 (aws/ebs ou CMK)
- Snapshots criptografados automaticamente

RDS:

- AES-256 com KMS
- Deve ser habilitado na criação
- Read replicas herdam criptografia

Encryption in Transit REDE

TLS/SSL:

- HTTPS para APIs AWS (obrigatório)
- TLS 1.2+ recomendado
- Certificate Manager (ACM) para certificados

VPN:

- Site-to-Site VPN (IPSec)
- Client VPN

Serviços específicos:

- S3: bucket policy para forçar HTTPS
"aws:SecureTransport": "false" → Deny
- RDS: `rds.force_ssl = 1`
- ElastiCache: in-transit encryption
- DynamoDB: sempre HTTPS (endpoint)

- ⚠ Sempre forçar HTTPS via policies
- ⚠ SSE-C requer HTTPS obrigatoriamente

API Gateway Authorizers (Detalhado)

Mecanismos de autenticação e autorização para APIs.

Lambda Authorizer CUSTOM

Fluxo:

1. Client envia request com token/params
2. API Gateway invoca Lambda Authorizer
3. Lambda valida e retorna IAM Policy
4. API Gateway cacheia policy (opcional)
5. Request é permitido ou negado

Retorno da Lambda:

```
{
  "principalId": "user123",
  "policyDocument": {
    "Statement": [{
      "Effect": "Allow",
      "Action": "execute-api:Invoke",
      "Resource": "arn:aws:execute-api:..."
    }]
  },
  "context": { "org": "acme" }
}
```

- Cache: 0 a 3600 segundos
- Custo: Lambda invocation + API Gateway

Cognito Authorizer JWT

Fluxo:

1. Client autentica no Cognito User Pool
2. Recebe JWT tokens (ID + Access)
3. Envia token no header Authorization
4. API Gateway valida JWT automaticamente
5. Sem invocação de Lambda

Configuração:

- Especificar User Pool ID
- Token source: header Authorization
- Validação: signature, expiration, issuer

Vantagens:

- Sem custo adicional (sem Lambda)
- Validação automática de JWT
- Integração nativa

Limitações:

- Apenas Cognito User Pool tokens
- Sem lógica customizada
- Sem acesso a headers/query params

IAM Authorization SIGV4

Fluxo:

1. Client assina request com SigV4
2. API Gateway valida assinatura
3. Verifica IAM policy do caller
4. Verifica resource policy da API

Quando usar:

- Chamadas entre serviços AWS
- Cross-account access
- Usuários IAM/Roles
- Não precisa de token externo

Configuração:

- Authorization: AWS4-HMAC-SHA256
- SDK assina automaticamente
- Resource policy para cross-account

- ⚠ Melhor para service-to-service
- ⚠ Não ideal para usuários finais (web/mobile)

 **Para o exame:** Lambda Authorizer = lógica customizada, tokens de terceiros. Cognito Authorizer = JWT do Cognito, sem Lambda. IAM Auth = service-to-service, SigV4. Para OAuth/OIDC de terceiros → Lambda Authorizer.

 **Developer — Domínio 03**

Implantação (Deployment)

CI/CD Pipeline, estratégias de deploy, SAM, CloudFormation, Elastic Beanstalk, ECS e ECR. Este domínio representa 24% do exame.

CI/CD Pipeline — AWS Developer Tools

Suite de serviços para automação completa do ciclo de desenvolvimento.

CodeCommit, CodeBuild, CodeDeploy, CodePipeline

CodeCommit [SOURCE](#)

- Repositório Git gerenciado pela AWS
- Privado, criptografado, altamente disponível
- Sem limite de tamanho de repositório
- Integração com IAM para acesso
- Triggers: SNS, Lambda (push events)
- Pull Requests com aprovações
- Cross-account access via IAM role

Autenticação:

- HTTPS: Git credentials ou credential helper
- SSH: SSH keys no IAM user
- Não suporta: anonymous access

CodeBuild**BUILD**

- Serviço de build serverless (pay-per-use)
- Compila código, roda testes, gera artefatos
- Ambientes: Docker containers gerenciados
- Escala automaticamente

buildspec.yml (na raiz do repo):

```
version: 0.2

env:
  variables:
    ENV: "prod"
  secrets-manager:
    DB_PASS: prod/db:password

phases:
  install:
    runtime-versions:
      nodejs: 18
  pre_build:
    commands:
      - npm install
  build:
    commands:
      - npm run build
      - npm test
  post_build:
    commands:
      - echo "Build complete"

artifacts:
  files:
    - '**/*'
  base-directory: dist

cache:
  paths:
    - node_modules/**/*
```

CodeDeploy DEPLOY

- Deploy automatizado para EC2, ECS, Lambda, On-premises
- Rollback automático em caso de falha
- Deployment groups: conjunto de instâncias

appspec.yml (EC2/On-premises):

version: 0.0

os: linux

files:

- source: /
destination: /var/www/html

hooks:

BeforeInstall:

- location: scripts/install_deps.sh
timeout: 300

AfterInstall:

- location: scripts/configure.sh

ApplicationStart:

- location: scripts/start_server.sh

ValidateService:

- location: scripts/health_check.sh

appspec.yml (Lambda):

version: 0.0

Resources:

- MyFunction:
Type: AWS::Lambda::Function
Properties:
Name: my-function
Alias: live
CurrentVersion: 1
TargetVersion: 2

Lifecycle Hooks (EC2):

ApplicationStop → DownloadBundle →

BeforeInstall → Install → AfterInstall →

ApplicationStart → ValidateService

CodePipeline **ORQUESTRAÇÃO**

- Orquestra o pipeline CI/CD completo
- Visual workflow com stages e actions
- Integra todos os Developer Tools

Stages típicos:

1. Source: CodeCommit, GitHub, S3, ECR
2. Build: CodeBuild, Jenkins
3. Test: CodeBuild, terceiros
4. Deploy: CodeDeploy, ECS, S3, CloudFormation
5. Approval: manual approval (SNS notification)

Características:

- Artifacts entre stages (S3)
- Parallel actions dentro de um stage
- Cross-region actions
- Cross-account deployments
- CloudWatch Events para triggers
- Webhook para GitHub

- ⚠ Cada stage deve ter pelo menos 1 action
- ⚠ Manual approval para gates de produção

 **Para o exame:** buildspec.yml = CodeBuild. appspec.yml = CodeDeploy. CodePipeline orquestra tudo. CodeBuild é serverless (sem gerenciar servidores). CodeDeploy hooks definem scripts em cada fase do deploy.

Estratégias de Deployment

Diferentes abordagens para atualizar aplicações em produção com diferentes trade-offs de risco e velocidade.

Estratégia	Downtime	Rollback	Custo	Descrição
All-at-Once	Sim	Re-deploy	Baixo	Atualiza todas as instâncias simultaneamente
Rolling	Parcial	Re-deploy	Baixo	Atualiza em batches (capacidade reduzida durante deploy)
Rolling with Additional Batch	Não	Re-deploy	Médio	Cria batch extra antes de atualizar (mantém capacidade)
Immutable	Não	Rápido (terminar novas)	Alto	Cria novas instâncias, troca quando saudáveis
Blue/Green	Não	Instantâneo (swap)	Alto (2x)	Ambiente novo completo, swap de DNS/LB
Canary	Não	Rápido	Médio	% pequeno do tráfego para nova versão, depois 100%
Linear	Não	Rápido	Médio	Incremento gradual do tráfego (10% a cada 10 min)

CodeDeploy – EC2 ESTRATÉGIAS

In-Place Deployment:

- AllAtOnce: todas de uma vez
- HalfAtATime: metade por vez
- OneAtATime: uma por vez
- Custom: definir % ou número

Blue/Green Deployment:

- Cria novo Auto Scaling Group
- Registra no Load Balancer
- Desregistra instâncias antigas
- Rollback: re-route para blue

CodeDeploy – Lambda**TRAFFIC SHIFTING**

- **Canary10Percent5Minutes:**
10% por 5 min, depois 100%
- **Canary10Percent10Minutes:**
10% por 10 min, depois 100%
- **Linear10PercentEvery1Minute:**
+10% a cada 1 minuto
- **Linear10PercentEvery10Minutes:**
+10% a cada 10 minutos
- **AllAtOnce:**
100% imediatamente

Pre/Post Traffic Hooks:

- Lambda functions para validação
- **BeforeAllowTraffic:** teste antes
- **AfterAllowTraffic:** teste depois
- Falha no hook = rollback automático



Para o exame: Blue/Green = rollback instantâneo (mais seguro, mais caro). Canary = teste com % pequeno antes. Immutable = novas instâncias (rollback rápido). Rolling = capacidade reduzida durante deploy.

AWS SAM (Serverless Application Model)

Framework para definir e deployar aplicações serverless. Extensão do CloudFormation.

SAM Template

TEMPLATE.YAML

```
AWSTemplateFormatVersion: '2010-09-09'
```

```
Transform: AWS::Serverless-2016-10-31
```

```
Globals:
```

```
  Function:
```

```
    Timeout: 30
```

```
    Runtime: python3.11
```

```
    MemorySize: 256
```

```
  Environment:
```

```
    Variables:
```

```
      TABLE_NAME: !Ref MyTable
```

```
Resources:
```

```
  MyFunction:
```

```
    Type: AWS::Serverless::Function
```

```
    Properties:
```

```
      Handler: app.handler
```

```
      CodeUri: src/
```

```
      Events:
```

```
        ApiEvent:
```

```
          Type: Api
```

```
          Properties:
```

```
            Path: /items
```

```
            Method: get
```

```
      Policies:
```

```
        - DynamoDBCrudPolicy:
```

```
          TableName: !Ref MyTable
```

```
  MyTable:
```

```
    Type: AWS::Serverless::SimpleTable
```

```
    Properties:
```

```
      PrimaryKey:
```

```
        Name: id
```

```
        Type: String
```

```
  MyApi:
```

```
    Type: AWS::Serverless::Api
```

```
    Properties:
```

```
      StageName: prod
```

Auth:

DefaultAuthorizer: MyCognitoAuth

SAM CLI **COMANDOS**

Desenvolvimento local:

```
$ sam init          # scaffold novo projeto
$ sam build         # compila e prepara artefatos
$ sam local invoke  # invoca Lambda localmente
$ sam local start-api # API Gateway local
$ sam local start-lambda # endpoint Lambda local
```

Deploy:

```
$ sam deploy --guided # deploy interativo
$ sam deploy          # deploy com samconfig.toml
```

Outros:

```
$ sam validate      # valida template
$ sam logs          # tail de logs do CloudWatch
$ sam sync          # sync rápido (dev)
```

samconfig.toml:

```
[default.deploy.parameters]
stack_name = "my-app"
s3_bucket = "my-deploy-bucket"
region = "us-east-1"
capabilities = "CAPABILITY_IAM"
confirm_changeset = true
```

⚠ Transform: AWS::Serverless-2016-10-31
é obrigatório no template SAM

SAM Globals e Policy Templates

Globals

HERANÇA

Globals:

Function:

Runtime: python3.11

Timeout: 30

MemorySize: 256

Tracing: Active

Environment:

Variables:

STAGE: prod

- Aplica a TODAS as funções do template
- Função individual pode sobrescrever
- Reduz repetição no template
- Suporta: Function, Api, HttpApi, SimpleTable

Policy Templates

PERMISSÕES

Policies simplificadas do SAM:

- DynamoDBCrudPolicy
- S3ReadPolicy / S3CrudPolicy
- SQSPollerPolicy
- SNSPublishMessagePolicy
- KMSDecryptPolicy
- LambdaInvokePolicy
- StepFunctionsExecutionPolicy
- CloudWatchPutMetricPolicy

Exemplo:

Policies:

- DynamoDBCrudPolicy:
 - TableName: !Ref MyTable
- S3ReadPolicy:
 - BucketName: !Ref MyBucket

⚠ Muito mais simples que IAM policies manuais

 **Para o exame:** SAM = extensão do CloudFormation para serverless. Transform é obrigatório. sam local para desenvolvimento local. Globals reduz repetição. Policy Templates simplificam IAM. sam build + sam deploy para CI/CD.

AWS CloudFormation

Infrastructure as Code — define e provisiona recursos AWS de forma declarativa.

Intrinsic Functions

Funções Principais

REF & GETATT

`!Ref LogicalName`

- Retorna valor do recurso ou parâmetro
- Recurso: geralmente ID físico
- Parâmetro: valor do parâmetro

`!GetAtt Resource.Attribute`

- Retorna atributo específico do recurso
- Ex: `!GetAtt MyBucket.Arn`
- Ex: `!GetAtt MyBucket.DomainName`

`!Sub "Hello ${AWS::StackName}"`

- Substitui variáveis em strings
- Suporta pseudo-parâmetros

`!Join [delimiter, [values]]`

- Concatena strings com delimitador

`!Select [index, [list]]`

- Seleciona item de uma lista

`!Split [delimiter, string]`

- Divide string em lista

Condicionalis & Mapping LÓGICA

Conditions:

```
IsProd: !Equals [!Ref Env, "prod"]
```

Resources:

ProdBucket:

```
Type: AWS::S3::Bucket
```

```
Condition: IsProd
```

```
!If [condition, true-value, false-value]
```

```
!Equals [value1, value2]
```

```
!Not [condition]
```

```
!And [conditions...]
```

```
!Or [conditions...]
```

Mappings:

RegionMap:

```
us-east-1:
```

```
AMI: ami-12345
```

```
eu-west-1:
```

```
AMI: ami-67890
```

```
!FindInMap [MapName, Key1, Key2]
```

- Ex: !FindInMap [RegionMap, !Ref AWS::Region, AMI]

Pseudo-parâmetros:

- AWS::AccountId, AWS::Region
- AWS::StackName, AWS::StackId
- AWS::NoValue

Change Sets, Nested Stacks e Drift Detection

Change Sets

PREVIEW

- Preview de mudanças antes de aplicar
- Mostra: Add, Modify, Remove
- Não garante sucesso (apenas preview)
- Workflow:
 1. Criar change set
 2. Revisar mudanças
 3. Executar ou deletar

Replacement:

- True: recurso será recriado
- False: update in-place
- Conditional: depende do atributo

⚠ Sempre usar change sets em produção

Nested Stacks

MODULARIZAÇÃO

- Stack dentro de stack
- Reutilização de templates
- Cada nested stack = recurso `AWS::CloudFormation::Stack`

Resources:

VPCStack:

Type: `AWS::CloudFormation::Stack`

Properties:

TemplateURL: `https://s3.../vpc.yaml`

Parameters:

CIDR: `"10.0.0.0/16"`

Cross-Stack References:

- Export: Output com Export Name
- Import: `!ImportValue` ExportName
- Diferente de nested (compartilha entre stacks independentes)

⚠ Nested = composição (lifecycle junto)

⚠ Cross-stack = compartilhamento (lifecycle separado)

Drift Detection COMPLIANCE

- Detecta mudanças manuais nos recursos
- Compara estado atual vs template
- Status: IN_SYNC, DRIFTED, NOT_CHECKED

Drift Status por recurso:

- MODIFIED: propriedades alteradas
- DELETED: recurso removido
- NOT_CHECKED: não verificado

Ações após detectar drift:

- Atualizar template para refletir mudanças
- Reverter mudanças manuais
- Import resource para gerenciar

- ⚠ Nem todos os recursos suportam drift detection
- ⚠ Não detecta drift em nested stacks automaticamente

 **Para o exame:** !Ref retorna ID, !GetAtt retorna atributos. Change Sets para preview. Nested Stacks para modularização. Cross-Stack para compartilhar outputs. Drift Detection para compliance.

AWS Elastic Beanstalk

PaaS que gerencia infraestrutura automaticamente — deploy de aplicações web sem gerenciar servidores.

Plataformas & Ambientes CONFIGURAÇÃO

Plataformas suportadas:

- Node.js, Python, Java, .NET, PHP, Ruby, Go
- Docker (single/multi-container)
- Custom platform (Packer)

Tipos de ambiente:

- Web Server Environment:
 - ELB + Auto Scaling + EC2
 - Para aplicações web (HTTP)
- Worker Environment:
 - SQS + Auto Scaling + EC2
 - Para processamento em background
 - Daemon processa mensagens da fila

Componentes:

- Application: container lógico
- Environment: instância da aplicação
- Application Version: código deployado
- Configuration: settings do ambiente

Deployment Policies

DEPLOY

- All at once: rápido, com downtime
- Rolling: batches, capacidade reduzida
- Rolling with additional batch: sem redução
- Immutable: novas instâncias, rollback rápido
- Traffic splitting: canary testing
- Blue/Green: swap de environment URLs

Rollback:

- All at once: re-deploy versão anterior
- Rolling: re-deploy
- Immutable: terminar novas instâncias
- Blue/Green: swap URLs de volta

- ⚠ Immutable e Blue/Green são os mais seguros
- ⚠ Traffic splitting = canary nativo do Beanstalk

.ebextensions CUSTOMIZAÇÃO

Diretório `.ebextensions/` na raiz do source bundle:

`.ebextensions/01-packages.config:`

`packages:`

`yum:`

`git: []`

`nginx: []`

`.ebextensions/02-files.config:`

`files:`

`"/etc/nginx/conf.d/proxy.conf":`

`mode: "000644"`

`content: |`

`proxy_pass http://localhost:8080;`

`.ebextensions/03-commands.config:`

`container_commands:`

`01_migrate:`

`command: "python manage.py migrate"`

`leader_only: true`

Ordem de execução:

- Arquivos `.config` em ordem alfabética
- `packages` → `sources` → `files` → `commands`
- `container_commands`: após app deploy
- `leader_only`: apenas em 1 instância



Para o exame: `.ebextensions` para customizar ambiente. Worker environment para SQS processing. Immutable deployment para rollback rápido. `leader_only` para migrations. Docker suportado nativamente.

Amazon ECS e ECR

Orquestração de containers Docker na AWS com opções de compute gerenciado (Fargate) ou self-managed (EC2).

Task Definitions BLUEPRINT

- JSON que define como rodar containers
- Imagem Docker, CPU, memória
- Port mappings, volumes
- Environment variables
- Logging configuration
- IAM roles

```
{
  "family": "my-app",
  "taskRoleArn": "arn:aws:iam::...",
  "executionRoleArn": "arn:aws:iam::...",
  "containerDefinitions": [{
    "name": "app",
    "image": "123456.dkr.ecr.us-east-1.amazonaws.com/my-app:latest",
    "cpu": 256,
    "memory": 512,
    "portMappings": [{"containerPort": 8080}],
    "logConfiguration": {
      "logDriver": "awslogs",
      "options": {
        "awslogs-group": "/ecs/my-app",
        "awslogs-region": "us-east-1"
      }
    }
  ]
}
```

Roles:

- Task Role: permissões do container (app)
- Execution Role: permissões do ECS agent (pull image, logs)

Fargate vs EC2

LAUNCH TYPE

Fargate (Serverless):

- Sem gerenciar instâncias EC2
- Paga por vCPU + memória por segundo
- Escala automaticamente
- Isolamento por task (segurança)
- Sem acesso ao host
- Ideal para: microservices, batch jobs

EC2 Launch Type:

- Você gerencia as instâncias EC2
- Mais controle (GPU, instance type)
- Pode ser mais barato em escala
- ECS Agent em cada instância
- Capacity Providers para Auto Scaling
- Ideal para: GPU, compliance, custo otimizado

ECS Services:

- Mantém N tasks rodando
- Load balancer integration (ALB/NLB)
- Auto Scaling (target tracking, step)
- Rolling update ou Blue/Green (CodeDeploy)
- Service discovery (Cloud Map)

Amazon ECR **REGISTRY**

Elastic Container Registry:

- Registry privado de Docker images
- Integração nativa com ECS, EKS, Lambda
- Criptografia at rest (KMS)
- Image scanning (vulnerabilidades)
- Cross-region e cross-account replication

Lifecycle Policies:

- Limpar imagens antigas automaticamente
- Por contagem ou por idade
- Manter apenas N imagens mais recentes

Comandos:

Login

```
aws ecr get-login-password | \
    docker login --username AWS --password-stdin \
    123456.dkr.ecr.us-east-1.amazonaws.com
```

Push

```
docker tag app:latest 123456.dkr.ecr.../app:latest
docker push 123456.dkr.ecr.../app:latest
```

Immutable tags: previne overwrite de tags

 **Para o exame:** Task Role = permissões da app. Execution Role = permissões do ECS agent. Fargate = serverless containers. ECR lifecycle policies para limpeza. ECS Service para manter tasks rodando com LB.

 **Developer — Domínio 04**

Troubleshooting e Otimização

CloudWatch, X-Ray, CloudTrail, caching, performance, debugging e observabilidade. Este domínio representa 18% do exame.

Amazon CloudWatch

Serviço de monitoramento e observabilidade — métricas, logs, alarmes e dashboards.

Metrics e Custom Metrics

Métricas Padrão BUILT-IN

EC2 (a cada 5 min, detailed: 1 min):

- CPUUtilization, NetworkIn/Out
- DiskReadOps, DiskWriteOps
- StatusCheckFailed

⚠ NÃO inclui: memória, disco (precisa agent)

Lambda:

- Invocations, Duration, Errors
- Throttles, ConcurrentExecutions
- IteratorAge (streams)

API Gateway:

- Count, Latency, IntegrationLatency
- 4XXError, 5XXError, CacheHitCount

DynamoDB:

- ConsumedReadCapacityUnits
- ConsumedWriteCapacityUnits
- ThrottledRequests
- SystemErrors

Custom Metrics

PUTMETRICDATA

- Enviar métricas customizadas via API
- PutMetricData API
- Resolução: Standard (60s) ou High (1s)
- Dimensões: até 30 por métrica
- Namespace customizado

```
import boto3
cw = boto3.client('cloudwatch')
cw.put_metric_data(
    Namespace='MyApp',
    MetricData=[{
        'MetricName': 'OrdersProcessed',
        'Value': 42,
        'Unit': 'Count',
        'Dimensions': [{
            'Name': 'Environment',
            'Value': 'prod'
        }]
    }]
)
```

Embedded Metric Format (EMF):

- Métricas dentro de logs estruturados
- Automaticamente extraídas como métricas
- Sem chamada PutMetricData separada
- Ideal para Lambda (custo menor)

CloudWatch Logs

Logs Structure

ORGANIZAÇÃO

Hierarquia:

- Log Group: /aws/lambda/my-function
 - └─ Log Stream: 2024/01/15/[\$LATEST]abc123
 - └─ Log Events: timestamp + message

Retention: 1 dia a 10 anos (ou never expire)

Encryption: KMS opcional

Fontes comuns:

- Lambda (automático)
- ECS (awslogs driver)
- EC2 (CloudWatch Agent)
- API Gateway (access/execution logs)
- VPC Flow Logs
- Route 53 DNS queries

Export:

- S3 (batch, até 12h delay)
- Kinesis Data Firehose (near real-time)
- Subscription Filters → Lambda, Kinesis, Firehose

Logs Insights QUERY

- Query language para análise de logs
- Busca em múltiplos log groups
- Visualizações (bar, line, stacked)

Exemplos de queries:

Top 20 erros

```
fields @timestamp, @message
| filter @message like /ERROR/
| sort @timestamp desc
| limit 20
```

Latência média por endpoint

```
filter @type = "REPORT"
| stats avg(@duration) as avgDuration
  by bin(5m)
```

Requests por status code

```
fields @timestamp
| filter ispresent(status)
| stats count(*) by status
```

Campos automáticos:

- @timestamp, @message, @logStream
- @log (log group)
- Lambda: @duration, @billedDuration, @memorySize, @maxMemoryUsed

Alarms e Dashboards

CloudWatch Alarms

ALERTAS

Estados:

- OK: métrica dentro do threshold
- ALARM: métrica excedeu threshold
- INSUFFICIENT_DATA: dados insuficientes

Configuração:

- Metric + Statistic (Average, Sum, Max, p99)
- Period: 10s, 30s, 60s, ou múltiplos de 60s
- Evaluation Periods: quantos períodos avaliar
- Datapoints to Alarm: quantos devem violar

Actions:

- SNS notification (email, SMS, Lambda)
- Auto Scaling action (scale out/in)
- EC2 action (stop, terminate, reboot)
- Systems Manager action

Composite Alarms:

- Combina múltiplos alarms (AND/OR)
- Reduz alarm noise
- Ex: CPU HIGH AND Memory HIGH

Dashboards VISUALIZAÇÃO

- Visualização customizada de métricas
- Cross-account e cross-region
- Auto-refresh (10s, 1m, 5m, 15m, 1h)
- Widgets: line, stacked area, number, gauge, text

Custo:

- 3 dashboards gratuitos (até 50 métricas)
- \$3/dashboard/mês adicional

Boas práticas:

- Dashboard por serviço/aplicação
- Incluir métricas de negócio
- Alarmes visíveis no dashboard
- Annotations para eventos importantes

 **Para o exame:** EC2 não monitora memória/disco nativamente (precisa CloudWatch Agent). EMF para métricas em Lambda (custo menor). Logs Insights para queries ad-hoc. Subscription Filters para streaming de logs. Composite Alarms para reduzir noise.

AWS X-Ray

Distributed tracing para análise e debug de aplicações distribuídas e microservices.

Conceitos Fundamentais

Tracing Components

ESTRUTURA

Trace:

- Representa uma request end-to-end
- Composto por múltiplos segments
- Trace ID: identificador único

Segment:

- Representa um serviço/recurso
- Contém: nome, ID, start/end time
- Status: OK, error, fault, throttle

Subsegment:

- Detalhe dentro de um segment
- Chamadas a AWS services, HTTP, SQL
- Permite granularidade fina
- Pode ser remoto (downstream service)

Service Map:

- Visualização gráfica dos serviços
- Mostra latência e erros entre serviços
- Identifica gargalos automaticamente

Annotations & Metadata DADOS

Annotations:

- Key-value pairs INDEXADOS
- Podem ser usados para filtrar traces
- Tipos: string, number, boolean
- Ideal para: user_id, order_id, environment

```
xray_recorder.put_annotation('user_id', '12345')  
xray_recorder.put_annotation('payment', 'credit')
```

Metadata:

- Key-value pairs NÃO indexados
- Qualquer tipo de dado (objetos, arrays)
- Não podem ser usados para filtrar
- Ideal para: request body, response, debug info

```
xray_recorder.put_metadata('response', data)
```

⚠ Annotations = filtráveis (busca)

⚠ Metadata = apenas visualização (debug)

Sampling Rules custo

- Controla quantas requests são traced
- Reduz custo e overhead

Regra padrão:

- Reservoir: 1 request/segundo (garantido)
- Rate: 5% das requests adicionais

Custom rules:

- Por service name, HTTP method, URL path
- Prioridade (menor número = maior prioridade)
- Pode ter reservoir e rate diferentes

Exemplo:

- /health: 0 reservoir, 0% rate (não traçar)
- /api/payments: 10 reservoir, 100% rate
- Default: 1 reservoir, 5% rate

- ⚠ Sampling reduz custo mas pode perder traces
- ⚠ Aumentar sampling para debugging

Integração com Serviços

Lambda + X-Ray SERVERLESS

Habilitar:

- Active tracing na configuração da função
- Ou via SAM: Tracing: Active

Permissões necessárias:

- AWSXRayDaemonWriteAccess na execution role

Variáveis de ambiente (automáticas):

- `_X_AMZN_TRACE_ID`: trace header
- `AWS_XRAY_DAEMON_ADDRESS`: daemon endpoint

SDK:

```
from aws_xray_sdk.core import patch_all
patch_all() # instrumenta boto3, requests, etc.
```

- ⚠ Lambda cria segment automaticamente
- ⚠ Você adiciona subsegments para detalhes

EC2 / ECS + X-Ray **DAEMON****EC2:**

- Instalar X-Ray Daemon (UDP port 2000)
- Daemon envia traces para X-Ray API
- IAM role com permissões de X-Ray

ECS:

- X-Ray daemon como sidecar container
- Ou como daemon service (1 por host)
- Task role com permissões de X-Ray

Fargate:

- Sidecar container obrigatório
- Não tem acesso ao host

API Gateway:

- Habilitar X-Ray tracing no stage
- Cria segment para cada request
- Mostra latência de integração

- ⚠ Daemon coleta e envia em batch
- ⚠ Sidecar pattern para containers

 **Para o exame:** Annotations são indexadas (filtráveis). Metadata não são indexadas. Sampling controla custo. Lambda: Active Tracing + execution role. ECS: sidecar container. Service Map mostra gargalos visualmente.

AWS CloudTrail

Auditoria e governança — registra todas as chamadas de API feitas na conta AWS.

Tipos de Eventos AUDITORIA

Management Events (padrão):

- Operações de gerenciamento de recursos
- CreateBucket, RunInstances, CreateUser
- Sempre habilitado (90 dias gratuito)
- Read events + Write events

Data Events (custo adicional):

- Operações nos dados dos recursos
- S3: GetObject, PutObject, DeleteObject
- Lambda: Invoke
- DynamoDB: GetItem, PutItem
- Alto volume → custo significativo

CloudTrail Insights:

- Detecta atividade incomum automaticamente
- Baseline de uso normal
- Alerta quando há desvio
- Ex: pico de API calls, erros incomuns

Configuração TRAIL

Trail:

- Entrega logs para S3 bucket
- Pode enviar para CloudWatch Logs
- Multi-region trail (recomendado)
- Organization trail (todas as contas)

Log File:

- JSON com detalhes da API call
- eventTime, eventSource, eventName
- userIdentity (quem fez)
- sourceIPAddress
- requestParameters
- responseElements

Integridade:

- Log file validation (digest files)
- Detecta se logs foram alterados/deletados
- S3 Object Lock para imutabilidade

- ⚠ CloudTrail = quem fez o quê e quando
- ⚠ CloudWatch = métricas e performance
- ⚠ X-Ray = tracing de requests distribuídas

🎯 **Para o exame:** CloudTrail = auditoria de API calls (quem, quando, o quê). Management events gratuitos (90 dias). Data events têm custo adicional. Insights detecta anomalias. Diferente de CloudWatch (performance) e X-Ray (tracing).

Estratégias de Caching

Diferentes camadas de cache para melhorar performance e reduzir custos.

ElastiCache: Redis vs Memcached

Característica	Redis	Memcached
Estruturas de dados	Strings, Lists, Sets, Hashes, Sorted Sets, Streams	Apenas strings (key-value simples)
Persistência	Sim (RDB snapshots, AOF)	Não (apenas memória)
Replicação	Sim (read replicas, Multi-AZ)	Não
Cluster Mode	Sim (sharding automático)	Sim (multi-node)
Pub/Sub	Sim	Não
Backup/Restore	Sim	Não
Multi-threaded	Não (single-threaded)	Sim
Caso de uso	Sessions, leaderboards, real-time analytics, queues	Cache simples, objetos grandes, multi-threaded

Caching Patterns

Lazy Loading (Cache-Aside) READ

Fluxo:

1. App verifica cache
2. Cache hit → retorna dados
3. Cache miss → busca no DB
4. Armazena no cache
5. Retorna dados

Vantagens:

- Apenas dados requisitados são cacheados
- Node failure não é fatal (busca do DB)
- Cache se preenche naturalmente

Desvantagens:

- Cache miss = 3 round trips (penalty)
- Dados podem ficar stale (desatualizados)
- Cold start: cache vazio inicialmente

Write-Through WRITE

Fluxo:

1. App escreve no cache E no DB
2. Cache sempre atualizado
3. Reads sempre do cache (fresh data)

Vantagens:

- Dados nunca stale no cache
- Sem cache miss penalty (dados já estão lá)

Desvantagens:

- Write penalty (2 writes por operação)
- Dados no cache podem nunca ser lidos
- Node failure = dados perdidos até re-write

Combinação ideal:

- Write-Through + Lazy Loading
- TTL para expirar dados não acessados

Outras Camadas de Cache

DAX DYNAMODB

- Cache in-memory para DynamoDB
- Latência de microssegundos
- API compatível com DynamoDB
- Write-through (automático)
- Item cache + Query cache
- Ideal para: read-heavy, hot partitions
- Não suporta strongly consistent reads

API Gateway Cache API

- Cache de respostas no API Gateway
- Apenas REST API (não HTTP API)
- TTL: 0-3600s (padrão: 300s)
- Tamanho: 0,5 GB a 237 GB
- Por stage, pode ser por method
- Invalidar: Cache-Control: max-age=0
- Custo por hora (não por request)

CloudFront CDN

- Cache na edge (global)
- TTL controlado por headers:
 - Cache-Control: max-age
 - Expires
- Invalidation: criar invalidation request
- Behaviors: cache por path pattern
- Origin Shield: camada extra de cache
- Ideal para: conteúdo estático, APIs globais

 **Para o exame:** Redis = mais recursos (persistência, replicação, pub/sub). Memcached = simples, multi-threaded. Lazy Loading = cache miss penalty. Write-Through = dados sempre fresh. DAX = DynamoDB cache (microsegundos).

Lambda Optimization

Técnicas para otimizar performance, custo e cold starts em funções Lambda.

Memory & Performance TUNING

Memória (128 MB - 10.240 MB):

- CPU é proporcional à memória
- Mais memória = mais CPU = execução mais rápida
- Pode ser mais barato com mais memória
(executa mais rápido, cobra menos tempo)

AWS Lambda Power Tuning:

- Ferramenta open-source (Step Functions)
- Testa função com diferentes memórias
- Encontra melhor custo-benefício
- Gráfico de custo vs duração

Boas práticas:

- Minimizar tamanho do pacote de deploy
- Usar layers para dependências compartilhadas
- Reutilizar conexões (keep-alive)
- Processar em batch quando possível

Cold Starts LATÊNCIA

Cold Start = primeira invocação (ou após inatividade):

1. Download do código
2. Inicializar runtime
3. Executar código de inicialização (fora do handler)
4. Executar handler

Fatores que aumentam cold start:

- Pacote grande (mais download)
- VPC (ENI attachment: +1-2s)
- Runtime pesado (Java > Python > Node)
- Muitas dependências

Soluções:

- Provisioned Concurrency (elimina cold start)
- Manter funções "warm" (ping periódico)
- Reduzir tamanho do pacote
- Inicializar SDK fora do handler
- Usar SnapStart (Java only)
- Evitar VPC se não necessário

Best Practices **CÓDIGO**

```
# FORA do handler (reutilizado entre invocações)
import boto3
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table(os.environ['TABLE_NAME'])

def handler(event, context):
    # DENTRO do handler (executado a cada invocação)
    item = table.get_item(Key={'id': event['id']})
    return item
```

Boas práticas:

- Conexões DB/SDK FORA do handler
- Variáveis de ambiente para config
- /tmp para cache de arquivos (512 MB - 10 GB)
- Idempotência (mesma request = mesmo resultado)
- Timeout < 30s para APIs (API Gateway limit)
- Usar X-Ray para identificar gargalos
- Async para operações não-críticas

 **Para o exame:** Inicializar SDK/conexões FORA do handler. Provisioned Concurrency elimina cold starts. Mais memória = mais CPU. SnapStart para Java. VPC aumenta cold start. Lambda Power Tuning para encontrar memória ideal.

DynamoDB Optimization

Técnicas avançadas de design de tabelas e otimização de performance.

Partition Key Design DISTRIBUIÇÃO

Boa partition key:

- Alta cardinalidade (muitos valores únicos)
- Distribuição uniforme de requests
- Evitar "hot partitions"

Exemplos bons:

- user_id (milhões de valores)
- order_id (UUID)
- device_id + timestamp

Exemplos ruins:

- status ("active"/"inactive" – 2 valores)
- date (todos acessam "hoje")
- country (poucos valores, distribuição desigual)

Capacidade por partição:

- 3000 RCU + 1000 WCU por partição
- Hot partition = throttling mesmo com capacidade sobrando

Write Sharding ANTI-PATTERN

Problema: hot partition (muitos writes na mesma PK)

Solução: Write Sharding

- Adicionar sufixo aleatório à partition key
- Distribuí writes entre múltiplas partições

Exemplo:

- Original PK: "2024-01-15" (hot!)
- Sharded PK: "2024-01-15_0", "2024-01-15_1", ..., "2024-01-15_9"
- 10x mais throughput

Trade-off:

- Reads precisam fazer Scatter-Gather
(query em todas as shards e agregar)
- Mais complexidade no código
- Ideal para: write-heavy com PK previsível

GSI Overloading SINGLE TABLE

Single Table Design:

- Múltiplas entidades na mesma tabela
- GSI com atributos genéricos (SK, GSI1PK, GSI1SK)
- Reduz número de tabelas e queries

Exemplo:

PK	SK	GSI1PK	GSI1SK
USER#123	PROFILE	EMAIL#a@b	USER#123
USER#123	ORDER#001	STATUS#open	2024-01-15
USER#123	ORDER#002	STATUS#done	2024-01-14
ORG#abc	MEMBER#123	USER#123	ORG#abc

Access patterns:

- Get user profile: PK=USER#123, SK=PROFILE
- Get user orders: PK=USER#123, SK begins_with ORDER#
- Get orders by status: GSI1PK=STATUS#open
- Get user's orgs: GSI1PK=USER#123, GSI1SK begins_with ORG#

- ⚠️ Complexo mas muito eficiente
- ⚠️ Planeje access patterns ANTES de criar a tabela

 **Para o exame:** Alta cardinalidade na partition key evita hot partitions. Write Sharding para distribuir writes. GSI Overloading para single-table design. Planejar access patterns antes de criar a tabela. DAX para hot keys de leitura.